Walmart Demand Forecasting

Current Repository Architecture, Decision Register, Model Specifications, and Presentation Defense Guide

Technical Presentation Guide

October 18, 2026

Contents

1 Executive summary **3**

2 Evidence boundary: what can and cannot be claimed **4**
2.1 What the repository establishes . . . 4
2.2 What the source code claims . . . 4
2.3 What remains experimentally unproven . . . 4

3 Business and forecasting problem **5**
3.1 Objective . . . 5
3.2 Forecasting unit . . . 5
3.3 Why CA_1 and 365 days? . . . 5
3.4 Why a 28-day horizon? . . . 6

4 End-to-end project process **6**

5 Data sources and preprocessing **6**
5.1 Required source files . . . 6
5.2 Data cleaning, filtering, and alignment . . . 7
5.3 Sales matrix . . . 7
5.4 Calendar features . . . 7
5.5 Lag features . . . 8
5.6 Rolling statistics . . . 8
5.7 Price features . . . 8
5.8 Actual feature dimensionality . . . 8
5.9 Configuration inconsistency . . . 8

6 Dataset windows and chronological splitting **9**
6.1 Sliding-window samples . . . 9
6.2 Time split . . . 9
6.3 Role of each partition . . . 9
6.4 Stronger validation design . . . 10

7 Model-selection rationale and architectures **10**
7.1 Current specification matrix . . . 10
7.2 Linear direct-forecast baseline . . . 11
7.2.1 Why it was selected . . . 11
7.2.2 Capabilities and limits . . . 11
7.2.3 Current training choices . . . 11
7.3 Encoder-only Transformer . . . 11
7.3.1 Processing path . . . 11

7.3.2 Why it was selected . . . 11
7.3.3 Causal-mask decision . . . 11
7.3.4 Current architectural choices . . . 12
7.4 Encoder-decoder Transformer . . . 12
7.4.1 Encoder . . . 12
7.4.2 Decoder . . . 12
7.4.3 Teacher forcing . . . 12
7.4.4 Autoregressive inference . . . 12
7.4.5 Why it may improve accuracy . . . 12
7.4.6 Exposure-bias risk . . . 12
7.4.7 Current choices . . . 13
7.5 DeepAR-style probabilistic LSTM . . . 13
7.5.1 Why it was selected . . . 13
7.5.2 Architecture . . . 13
7.5.3 Current choices . . . 13
7.5.4 Limitations . . . 13
7.6 Compact Temporal Fusion Transformer . . . 13
7.6.1 Why it was selected . . . 14
7.6.2 Architecture . . . 14
7.6.3 Current choices . . . 14
7.6.4 Limitations . . . 14

8 Training decisions 14
8.1 Optimizer . . . 14
8.2 Training objectives . . . 14
8.3 Gradient clipping . . . 15
8.4 Learning-rate schedules . . . 15
8.5 Checkpointing . . . 15
8.6 Early stopping and warm-up caveats . . . 15
8.7 Reproducibility decision . . . 15
8.8 Sequential execution decision . . . 15

9 Evaluation metrics 15
9.1 Mean Absolute Error . . . 15
9.2 Root Mean Squared Error . . . 16
9.3 Mean Absolute Percentage Error . . . 16
9.4 RMSSE . . . 16
9.5 WRMSSE . . . 16
9.6 Consolidated experiment reporting . . . 16

10 Rigorous model-selection protocol 16
10.1 Stage 1: Baselines . . . 16
10.2 Stage 2: Architecture screening . . . 17
10.3 Stage 3: Focused tuning . . . 17
10.4 Stage 4: Freeze the system . . . 17
10.5 Stage 5: Final test . . . 17

11 Hyperparameter-tuning status and plan 17
11.1 What tuning was actually used . . . 17
11.2 Search method . . . 17
11.3 Shared search space . . . 17
11.4 Transformer search space . . . 18
11.5 DeepAR search space . . . 18
11.6 TFT search space . . . 18
11.7 Trial pruning and final confirmation . . . 18
11.8 Objective selection . . . 19

12 Complete decision register **19**
12.1 Data decisions . . . 19
12.2 Validation decisions . . . 19
12.3 Modeling decisions . . . 19
12.4 Operational decisions . . . 19

13 Known limitations and next improvements **20**

14 Presentation structure **20**

15 Cross-question defense guide **21**
15.1 Why use deep learning if a linear model is available? . . . 21
15.2 Why not use ARIMA? . . . 21
15.3 Why exclude XGBoost and LightGBM? . . . 21
15.4 Why use only 28 days of history? . . . 21
15.5 Why choose 7-, 14-, and 28-day lags? . . . 21
15.6 Why not include a 365-day lag? . . . 21
15.7 Why MAE instead of MSE? . . . 21
15.8 Why not use MAPE? . . . 21
15.9 Why use Adam? . . . 22
15.10 Why might the encoder-decoder model win? . . . 22
15.11 What is teacher forcing? . . . 22
15.12 How is target leakage prevented? . . . 22
15.13 Why use a causal encoder mask? . . . 22
15.14 How does the project represent uncertainty? . . . 22
15.15 Is Gaussian demand appropriate? . . . 22
15.16 Is the TFT interpretable? . . . 22
15.17 Does the model use known future holidays or prices? . . . 22
15.18 How should tuning avoid test leakage? . . . 22
15.19 How can we know whether an improvement is statistically real? . . . 23
15.20 Would the lowest-MAE model automatically be deployed? . . . 23
15.21 How would new products be handled? . . . 23
15.22 How would drift be monitored? . . . 23
15.23 What is the single most important next experiment? . . . 23

16 Repository verification notes **23**

1 Executive summary

This project is an end-to-end PyTorch framework for forecasting the next 28 days of unit sales for Walmart item-store time series from the M5 dataset. It loads daily sales, calendar information, and weekly prices; engineers temporal and commercial features; creates chronological training, validation, and test windows; and compares five forecasting architectures under a common pipeline.

The five candidate models are:

1. A linear direct-forecast baseline.
2. An encoder-only Transformer.
3. An autoregressive encoder-decoder Transformer.
4. A DeepAR-style probabilistic LSTM.
5. A compact Temporal Fusion Transformer (TFT).

The central research question is:

> Does additional temporal modeling capacity improve 28-day retail demand forecasts enough to justify its added training, inference, and operational complexity?

The current default experiment deliberately uses only store `CA_1` and its latest 365 days. This scope was chosen to make local laptop execution practical while retaining thousands of related item series

and a complete annual cycle. The filtering occurs before feature construction, which avoids allocating the full ten-store tensor.

The project is strongest as a modular experimental framework. It contains working implementations, a shared feature pipeline, chronological data splitting, configurable training, model-specific checkpointing, metrics, a one-click Windows launcher, and a self-contained HTML experiment report. All 29 repository tests pass. However, trained checkpoints and completed real-data results are generated locally and excluded from Git. Model-performance claims in source comments must therefore be presented as reference or previously reported values until the current all-model run produces its own report.

2 Evidence boundary: what can and cannot be claimed

2.1 What the repository establishes

The repository establishes that:

- All five model families are implemented and constructible from YAML configurations.
- The data loader produces chronologically ordered input and target windows.
- Model outputs have the expected 28-day shape.
- Gradients flow through all trainable architectures.
- DeepAR produces a strictly positive estimated scale.
- Training supports Adam, gradient clipping, learning-rate scheduling, validation, and checkpointing.
- MAE, RMSE, MAPE, RMSSE, and a bottom-level WRMSSE utility are implemented.
- Store and history filtering are tested for correct calendar and price alignment.
- The all-model runner records metrics, histories, timings, configurations, and embedded charts in one HTML report.
- The test suite passes 29 tests.

The tests validate software mechanics, tensor shapes, gradient flow, temporal target ordering, split construction, and metric arithmetic. They do not establish real-world forecasting accuracy, statistical significance, business value, uncertainty calibration, or superiority over external baselines.

2.2 What the source code claims

Source comments identify the encoder-decoder Transformer as the best-performing architecture and quote approximate MAE values:

Model	MAE quoted in source comments
Linear baseline	137.01
Encoder-only Transformer	134.73
Encoder-decoder Transformer	122.01

These figures are not accompanied by checkpoints, experiment metadata, logs, random seeds, data hashes, or a result table. The defensible wording is:

> The project documentation contains previously reported or reference MAE values that favor the encoder-decoder Transformer, but the current repository does not include sufficient artifacts to reproduce or independently verify them.

Do not state that these figures were reproduced unless the full experiments are run and saved.

2.3 What remains experimentally unproven

The repository does not yet establish that:

- The encoder-decoder Transformer wins under a fresh, controlled run.
- The supplied hyperparameters are optimal.
- Performance is stable across random seeds and forecast origins.
- The models outperform seasonal-naive, LightGBM, XGBoost, or classical statistical baselines.
- The evaluation matches the official hierarchical M5 protocol.
- The probabilistic predictions are calibrated.
- The accuracy improvement justifies the computational cost.

3 Business and forecasting problem

3.1 Objective

The system predicts daily unit demand for the next 28 days. Accurate forecasts support:

- Inventory replenishment.
- Safety-stock planning.
- Shelf availability.
- Promotion planning.
- Distribution and labor planning.
- Reducing stockouts and excess inventory.

The cost of forecast error is asymmetric in practice. Under-forecasting can produce lost sales and lower customer satisfaction, while over-forecasting ties up capital and can produce markdown or waste costs. The current training objective does not explicitly encode this asymmetry; it treats positive and negative absolute errors equally.

3.2 Forecasting unit

The lowest-level M5 series represents a particular item sold at a particular store. In the standard evaluation file, the data contains approximately 30,490 bottom-level series and 1,941 observed days. The implemented default selects the approximately 3,049 item series belonging to `CA_1` and retains the latest 365 days.

The project trains global models within the selected store: one shared set of model parameters learns from every retained `CA_1` item series. This is preferable to training thousands of unrelated models when individual series are short, sparse, or noisy. The model can learn common weekly, event, price, and volatility patterns across products.

The current pipeline does not provide explicit item, store, category, department, or state identifiers to the models. Series identity is inferred only from recent sales and engineered features. Categorical embeddings are therefore an important extension.

3.3 Why CA_1 and 365 days?

The original full-data design generated roughly 45.6 million training windows and very large in-memory tensors. That workload is unsuitable for an ordinary laptop. Selecting one store cuts the series count by about 90%; retaining 365 days limits feature memory and still represents a complete annual period.

This is a computational design decision, not a claim that `CA_1` represents every Walmart store. Conclusions from the current experiment are store-specific and should later be checked on other stores or the complete hierarchy. Filtering is performed before calendar tiling, lag creation, rolling statistics, and price construction, so the resource savings are real rather than cosmetic.

3.4 Why a 28-day horizon?

The 28-day horizon matches the M5 competition design and represents four operational weeks. It is long enough to test accumulated multi-step error and short enough to remain relevant for retail planning.

The input window is also 28 days. This captures four weekly cycles and makes the task four-weeks-in/four-weeks-out. It is a sensible initial value, not a demonstrated optimum. Longer histories may improve annual-seasonality and trend modeling and should be tuned.

4 End-to-end project process

The implemented workflow is:

1. Obtain the three M5 source files locally.
2. Filter sales and prices to `CA_1` before feature construction.
3. Retain the latest 365 sales days and align calendar rows to those exact day labels.
4. Restrict prices to the weeks required by the retained dates.
5. Build calendar, lag, rolling, and price features.
6. Split each series chronologically into train, validation, and test periods.
7. Generate overlapping 28-day input and 28-day target windows.
8. Build each architecture from its YAML configuration.
9. Train with Adam and the architecture-appropriate loss.
10. Evaluate validation MAE after every epoch.
11. Save each model's best and final checkpoints in a separate directory.
12. Load the best checkpoint for held-out inference.
13. Calculate MAE and RMSE, collect histories and timings, and embed comparison charts in `experiment_report.html`.

The intended model-selection process is a complexity ladder:

Stage	Candidate	Question being tested
1	Linear baseline	Do engineered features already explain most predictable variation?
2	Encoder-only Transformer	Do nonlinear and nonlocal historical interactions improve forecasts?
3	Encoder-decoder Transformer	Does autoregressive generation improve multi-horizon structure?
4	DeepAR	Does modeling forecast uncertainty add value?
5	Compact TFT	Do variable selection, recurrence, gating, and attention provide a better hybrid?

This structure is defensible because each model adds a specific capability rather than merely increasing size.

5 Data sources and preprocessing

5.1 Required source files

File	Role
`sales_train_evaluation.csv`	Daily unit sales for each item-store series
`calendar.csv`	Dates, weekdays, events, SNAP indicators, and week identifiers
`sell_prices.csv`	Weekly item prices by store

The data is intentionally excluded from Git due to size and distribution constraints.

5.2 Data cleaning, filtering, and alignment

The project performs targeted structural cleaning rather than a large generic imputation pipeline:

1. It reads sales, calendar, and price CSVs with pandas.
2. It filters sales and prices to `store_id == "CA_1"`.
3. It raises an error if the requested store does not exist.
4. It identifies sales columns by the `d_` prefix and sorts them numerically, preventing lexicographic errors such as placing `d_100` before `d_20`.
5. It keeps the final 365 day columns and raises an error if insufficient history exists.
6. It filters calendar rows to the exact retained day labels, restores their chronological order through an explicit day-position map, and verifies that every sales day has a calendar row.
7. It filters prices to `CA_1` and to the `wm_yr_wk` values present in the retained calendar.
8. It converts the resulting sales and features to 32-bit floating point for lower memory use and PyTorch compatibility.

No explicit outlier removal is performed because extreme retail demand can represent real promotions or events. Raw zero sales are preserved. Missing item-price histories result in zero-filled price features, and the initial portions of lag and rolling features are also zero-filled when insufficient prior history exists. These are implementation choices that should be evaluated against explicit missingness indicators or learned imputation.

5.3 Sales matrix

Columns beginning with `d_` are extracted into a floating-point matrix:

`(number of series, number of observed days)`

Each row is one item-store time series. The raw sales history is used both as a model input and to derive lag and rolling features.

5.4 Calendar features

Seven calendar variables are created:

- Weekday number.
- Month.
- California SNAP indicator.
- Texas SNAP indicator.
- Wisconsin SNAP indicator.
- Weekend indicator.
- Event-present indicator.

The reasoning is that retail demand responds to weekly shopping cycles, broad seasonal changes, assistance-program timing, weekends, and special events.

An important limitation is that named event categories are compressed into one binary has_event flag. Christmas, Easter, sporting events, and cultural events can have very different effects. A richer implementation should encode event names and types separately.

5.5 Lag features

The code creates lags of 7, 14, and 28 days.

- Lag 7 represents the same weekday one week earlier.
- Lag 14 measures persistence over two weeks.
- Lag 28 represents the equivalent weekday four weeks earlier.

These are domain-informed choices based on strong weekly retail seasonality. Lags of 1, 2, 3, 56, and 364/365 days are reasonable tuning candidates.

5.6 Rolling statistics

For 7-, 14-, and 28-day windows, the pipeline creates:

- Rolling mean.
- Rolling standard deviation.

The mean estimates local demand level, while standard deviation estimates recent volatility. Multiple windows distinguish short-term changes from stable behavior. Each day uses preceding values, not its current target value.

5.7 Price features

Two price features are produced:

- Normalized price: current price divided by the product-store mean price.
- Seven-day price momentum: current normalized price minus normalized price seven days earlier.

Normalization makes price movements more comparable across products. Momentum acts as a simplified price-change or promotion signal.

There is a leakage concern: mean price is calculated across all retained 365 days before the chronological split. A stricter design would fit normalization statistics on the 292-day training portion and reuse them for validation and test data. Alternatively, if the annual reference price is treated as known product metadata, that assumption must be explicitly documented.

5.8 Actual feature dimensionality

The feature engineer creates:

Group	Count
Calendar	7
Lags	3
Rolling means and standard deviations	6
Price	2
Total engineered features	18
Raw sales appended at model input	1
Total model input variables per time step	19

5.9 Configuration inconsistency

The YAML files expose `features`, `lag_features`, and `rolling_windows`, but the preprocessing class currently uses hard-coded constants. Consequently:

- Changing feature names in YAML does not change the generated tensor.
- Changing `lag_features` does not change the lag calculation.

- Changing `rolling_windows` does not change the rolling calculation.
- YAML lists 7- and 28-day rolling windows, while the code also produces a 14-day window.

This should be presented as configuration plumbing still to be completed, not as dynamic feature tuning already implemented.

6 Dataset windows and chronological splitting

6.1 Sliding-window samples

Every valid forecast origin creates one sample:

```
x_sales:    28 historical sales values
x_features: 28 x 18 engineered-feature values
y:          next 28 sales values
```

For a series with `D` days, the number of valid windows is:

```
D - input_window - forecast_horizon + 1
```

Overlapping windows improve data utilization and expose models to many seasonal conditions. Their disadvantage is high sample correlation. Evaluation across overlapping windows is a rolling-origin average, not a collection of fully independent cases.

For the expected 3,049 `CA_1` series and 365 retained days, the implemented 80/10/10 split gives:

Partition	Raw days	Windows per series	Approximate total samples
Training	292	237	722,613
Validation	36 plus 28 context days	9	27,441
Test	37 plus 28 context days	10	30,490

The exact series and sample counts are recorded automatically in the generated experiment report.

6.2 Time split

The split is chronological:

- First 80%: training.
- Next 10%: validation.
- Final 10%: testing.

Randomly shuffling time steps would leak future behavior into training, so chronological division is mandatory.

Validation and test partitions receive 28 days of context from the preceding period. These values serve only as model inputs; targets remain within the correct partition.

6.3 Role of each partition

- Training data fits model parameters.
- Validation data selects architecture, hyperparameters, learning rate behavior, and checkpoint.
- Test data provides the final unbiased estimate after every choice is frozen.

Repeatedly viewing test results and changing the system converts the test set into another validation set.

6.4 Stronger validation design

A single split may favor or punish models because of unusual conditions in one period. A stronger protocol uses expanding-window or rolling-origin validation:

```
Fold 1: train through T1, validate on the next 28 days
Fold 2: train through T2, validate on the next 28 days
Fold 3: train through T3, validate on the next 28 days
```

Hyperparameters should be selected on mean validation performance, with variability across folds and random seeds also reported.

7 Model-selection rationale and architectures

7.1 Current specification matrix

All models receive 28 time steps, 19 variables per time step, and produce a 28-day forecast. Approximate parameter counts are calculated from the current implementations.

Model	Core specification	Approx. parameters	Output style
Linear	Flattened 28 x 19 input; one 532-to-28 layer	14,924	Direct point forecast
Encoder-only	`d_model`=128; 8 heads; 3 encoder layers; `d_ff`=512	698,012	Direct point forecast
Encoder-decoder	`d_model`=128; 8 heads; 3 encoder + 3 decoder layers; `d_ff`=512	1,392,001	Autoregressive point forecast
DeepAR-style	Hidden size 128; 2 encoder/decoder LSTM layers	407,810	Gaussian mean and scale
Compact TFT	Hidden size 128; 4 heads; 1 LSTM layer; `d_ff`=256	519,064	Direct point forecast

Model	Epochs	Batch	Learning rate	Scheduler	Patience	Dropout
Linear	50	256	0.001	Reduce on plateau	5	N/A
Encoder-only	100	128	0.0001	Cosine	10	0.1
Encoder-decoder	100	128	0.0001	Cosine	10	0.1
DeepAR-style	100	128	0.0005	Reduce on plateau	10	0.1
Compact TFT	100	128	0.0001	Cosine	10	0.1

Data-loader workers are set to zero for reliable Windows/laptop execution. Device selection is automatic: CUDA is used when available; otherwise training runs on CPU.

7.2 Linear direct-forecast baseline

The baseline flattens all 28 historical time steps and 19 variables, then applies one linear layer to produce all 28 future values.

`28 x 19 = 532 inputs -> 28 forecast outputs`

With 19 inputs per step, it has approximately 14,924 trainable parameters.

7.2.1 Why it was selected

- Establishes the value added by complexity.
- Is fast, low-memory, and easy to debug.
- Can reveal data-pipeline problems before expensive training.
- Can be preferable in production if more complex models improve accuracy only marginally.

7.2.2 Capabilities and limits

It learns separate linear effects from every historical feature and position to every forecast horizon. It cannot directly express nonlinear price responses, interactions, regime changes, or complex temporal dependencies.

7.2.3 Current training choices

- 28-day input and output.
- Batch size 256.
- Learning rate 0.001.
- Adam optimizer.
- Reduce-on-plateau scheduler.
- 50 epochs.

A relatively large batch and learning rate are reasonable because the model is small and stable.

7.3 Encoder-only Transformer

Approximate parameter count: 698,012.

7.3.1 Processing path

1. Concatenate sales with engineered features.
2. Project the 19 variables to a 128-dimensional representation.
3. Add fixed sinusoidal positional encoding.
4. Apply layer normalization.
5. Process the sequence through three Transformer encoder layers.
6. Flatten all encoded positions.
7. Project directly to 28 forecast values.

7.3.2 Why it was selected

Self-attention lets the model learn nonlinear relationships between historical time positions. It can learn whether last week's matching weekday is more informative than yesterday, or whether price changes alter the relevance of recent demand.

7.3.3 Causal-mask decision

The encoder uses a causal mask, so each historical position cannot attend to later historical positions. This enforces strictly forward temporal representations. However, all 28 input days are known at

the forecast origin, so unmasked historical attention would not leak future targets. Masked versus unmasked encoding is therefore an experimental decision.

7.3.4 Current architectural choices

- Embedding size 128.
- Eight attention heads, giving 16 dimensions per head.
- Three encoder layers.
- Feed-forward size 512, or four times the embedding size.
- Dropout 0.1.
- Batch size 128.
- Learning rate 0.0001.
- Cosine learning-rate decay.
- 100 epochs.

These are conventional medium-sized Transformer defaults for a short sequence.

7.4 Encoder-decoder Transformer

Approximate parameter count: 1,392,001. The project configuration labels this architecture as the best model.

7.4.1 Encoder

The encoder receives 28 days of sales and engineered features, maps them to 128-dimensional embeddings, adds positional encodings, and processes them with three self-attention layers.

7.4.2 Decoder

The decoder uses masked self-attention over the forecast sequence and cross-attention over encoder memory. It predicts future values autoregressively.

7.4.3 Teacher forcing

During training, the decoder input is:

```
[last observed sale, true target day 1, ..., true target day 27]
```

It predicts target days 1 through 28. Teacher forcing makes training parallel and stable because the model receives correct previous values.

7.4.4 Autoregressive inference

At inference time, true future values are unavailable. The model predicts day 1, feeds that prediction into the next step, and continues until day 28.

7.4.5 Why it may improve accuracy

Unlike a direct projection, the decoder explicitly models dependencies inside the forecast horizon. Day 15 can depend on generated days 1 through 14 as well as the historical context.

7.4.6 Exposure-bias risk

Training uses true previous values, while inference uses the model's imperfect values. Errors may compound across the horizon. Scheduled sampling, input noise, direct multi-horizon decoding, or horizon-weighted loss can reduce this mismatch.

7.4.7 Current choices

- Embedding size 128.
- Eight heads.
- Three encoder and three decoder layers.
- Feed-forward size 512.
- Dropout 0.1.
- Batch size 128.
- Learning rate 0.0001.
- Cosine decay.
- 100 epochs.

7.5 DeepAR-style probabilistic LSTM

Approximate parameter count: 407,810.

7.5.1 Why it was selected

Retail planning needs uncertainty, not only a point estimate. A probabilistic model can support prediction intervals, stockout risk, safety stock, and exceedance probabilities.

7.5.2 Architecture

1. An LSTM encoder processes historical sales and covariates.
2. An LSTM decoder generates the future autoregressively.
3. A two-value output head predicts a Gaussian mean and raw scale.
4. Softplus plus a small constant makes scale strictly positive.
5. Training minimizes Gaussian negative log-likelihood.

7.5.3 Current choices

- Hidden size 128.
- Two LSTM layers.
- Dropout 0.1.
- Batch size 128.
- Learning rate 0.0005.
- Reduce-on-plateau scheduler.
- 100 epochs.

7.5.4 Limitations

This is DeepAR-style rather than a complete reproduction of the original system. Raw sales are nonnegative integer counts, often zero and overdispersed. Gaussian likelihood can produce negative means and may represent sparse demand poorly. Poisson, negative-binomial, or zero-inflated negative-binomial likelihoods should be tested.

The model estimates scale, but the current evaluation consumes only the mean. It does not calculate prediction intervals, CRPS, interval coverage, or calibration.

7.6 Compact Temporal Fusion Transformer

Approximate parameter count: 519,064.

7.6.1 Why it was selected

TFT is designed for multi-horizon forecasting with mixed covariates. It combines local sequence processing, variable selection, attention, residual connections, and gating.

7.6.2 Architecture

1. Each scalar feature has its own projection into the hidden space.
2. A softmax selection layer assigns time-dependent feature weights.
3. A gated residual network transforms the selected representation.
4. An LSTM learns local temporal structure.
5. Multi-head self-attention learns broader temporal relationships.
6. Further gating and feed-forward processing fuse the representation.
7. A direct projection produces 28 forecasts.

7.6.3 Current choices

- Hidden dimension 128.
- Four attention heads, giving 32 dimensions per head.
- One LSTM layer.
- Feed-forward size 256.
- Dropout 0.1.
- Batch size 128.
- Learning rate 0.0001.
- Cosine scheduler.
- 100 epochs.

7.6.4 Limitations

This is a compact TFT-inspired model, not the complete published TFT architecture. It lacks static-covariate encoders, static enrichment, explicit known-future processing, quantile loss, a future decoder, and exposed variable/attention weights. It should be described as a compact or TFT-style implementation.

8 Training decisions

8.1 Optimizer

All models use Adam. Adam provides adaptive per-parameter learning rates and is a robust default for attention and recurrent architectures.

The configuration contains an `optimizer` field, but the trainer always constructs Adam. The field is currently informational or unfinished.

8.2 Training objectives

The linear, encoder-only, encoder-decoder, and TFT models minimize mean absolute error:

```
MAE = mean(abs(prediction - actual))
```

MAE is directly interpretable in sales units and less dominated by extreme spikes than MSE.

DeepAR minimizes Gaussian negative log-likelihood because it learns both location and uncertainty. The console deliberately labels this value `Train loss`, while validation remains MAE on the predicted mean; this avoids incorrectly presenting DeepAR's training objective as MAE.

8.3 Gradient clipping

The global gradient norm is limited to 1.0. This reduces exploding gradients in recurrent and autoregressive models and is a conservative stability measure.

8.4 Learning-rate schedules

Reduce-on-plateau watches validation loss and halves the learning rate after the configured patience. It is used for the linear model and DeepAR.

Cosine annealing smoothly decreases the rate over the configured epochs. It is used for the Transformer models.

8.5 Checkpointing

After each epoch, validation MAE is calculated. If it is the lowest observed value, model weights are written to `best_model.pt`. Final weights are also written to `final_model.pt`.

Evaluation should use the best checkpoint rather than assuming the final epoch is best.

The all-model runner isolates weights under `checkpoints/<model-name>/`, preventing one architecture from overwriting another. Each directory contains `best_model.pt` and `final_model.pt`.

8.6 Early stopping and warm-up caveats

The `patience` setting controls only the reduce-on-plateau scheduler; training does not stop early.

Transformer YAML files specify warm-up steps, but the trainer does not implement warm-up. Cosine decay begins immediately.

8.7 Reproducibility decision

The consolidated runner sets Python, NumPy, PyTorch, and CUDA seeds to 42 before data/model execution and again before each model. This improves comparability of one run. It does not force deterministic CUDA algorithms, and a single seed is not enough to establish statistical stability. Final comparisons should use at least three independent seeds and report mean and standard deviation. The standalone single-model training script does not currently set seeds.

8.8 Sequential execution decision

The five models run sequentially rather than concurrently. This prevents multiple large models from competing for laptop RAM, GPU memory, disk bandwidth, and CPU workers. The feature dataset and temporal splits are constructed once and reused, ensuring that every model sees identical samples. A failed model is recorded in the report and does not prevent subsequent models from being attempted.

The Windows launcher automates environment preparation, data-file verification, dependency installation, tests, duplicate-process detection, and hidden background execution. Training logs are diagnostic artifacts; the primary review artifact is the consolidated HTML report. The launcher records a process ID and avoids starting a second run while the first is active.

9 Evaluation metrics

9.1 Mean Absolute Error

MAE answers: on average, how many units away is the forecast? It is the checkpoint-selection metric and the simplest business-facing measure.

Its limitation is that errors from high-volume series can dominate a sample-level average.

9.2 Root Mean Squared Error

RMSE squares errors before averaging, so large misses receive disproportionate weight. Comparing MAE and RMSE helps identify whether a model occasionally fails badly.

9.3 Mean Absolute Percentage Error

MAPE is implemented but is unsuitable as the primary metric because M5 sales frequently equal zero. Division by values near zero creates unstable percentages. Scaled error measures are more appropriate.

9.4 RMSSE

Root Mean Squared Scaled Error divides each series' forecast MSE by its historical one-step naive MSE. It is scale-independent and allows comparison between high- and low-volume products.

An RMSSE near or below 1 generally indicates performance comparable to or better than the scaling naive forecast, subject to the exact evaluation setup.

9.5 WRMSSE

Weighted RMSSE is central to M5 evaluation, but the current implementation is not the full official scorer. It defaults to uniform bottom-level weights, does not calculate revenue weights, and does not aggregate all hierarchy levels.

The evaluation script also calculates training history but does not pass it into the metric function. It therefore prints only MAE and RMSE today. RMSSE and WRMSSE are available utilities, not current CLI outputs.

9.6 Consolidated experiment reporting

The all-model runner evaluates each best checkpoint on the same test dataset and writes one self-contained `experiment_report.html`. The file embeds:

- Model status, test MAE, test RMSE, and best validation MAE.
- Parameter counts and elapsed training time.
- Cross-model MAE and RMSE bar charts.
- Validation-loss and learning-rate curves.
- Held-out forecast-versus-actual plots.
- Complete per-epoch histories and effective configurations.
- Dataset scope, split counts, seed, start/finish timestamps, and failures.

Images are encoded directly inside the HTML, so the report is portable as one human-readable artifact. Binary weights remain separate because embedding multiple model state dictionaries would make the report impractically large. The report is atomically rewritten after each model, preserving partial results if a later architecture fails.

10 Rigorous model-selection protocol

10.1 Stage 1: Baselines

Establish at least:

- Last-value naive.
- Seven-day seasonal naive.
- Four-week seasonal average.
- Linear direct model.

- LightGBM or XGBoost using the same lag features.

Complex models should not proceed on reputation alone; they must beat strong simple baselines.

10.2 Stage 2: Architecture screening

Train all five candidates using identical:

- Data cutoffs.
- Features.
- Validation origins.
- Random seeds.
- Search budgets.
- Evaluation metrics.

Record validation error, parameter count, memory, training time, and inference latency.

10.3 Stage 3: Focused tuning

Use a coarse search to eliminate weak families, then allocate larger tuning budgets to the best two or three architectures.

10.4 Stage 4: Freeze the system

Freeze feature definitions, transformations, architecture, hyperparameters, checkpoint rule, and seed policy before opening the final test set.

10.5 Stage 5: Final test

Evaluate once on the locked test period. Report overall and segmented performance by horizon, state, store, category, intermittency, and demand volume.

11 Hyperparameter-tuning status and plan

11.1 What tuning was actually used

No automated grid, random, Bayesian, or population-based hyperparameter search is implemented. The current YAML values are architecture-informed manual starting points. Model selection during a run consists of comparing five fixed configurations on the same split and selecting the lowest validation-MAE checkpoint within each configuration. Learning-rate schedulers adapt optimization during training, but they are not hyperparameter-search algorithms. The runner's `--epochs` option is an execution override for short tests, not a tuning system.

This distinction is important in a presentation: it is accurate to say that hyperparameters were configured and validation checkpoints were selected; it is not accurate to say that the repository has already optimized them.

11.2 Search method

Random or Bayesian search is preferable to a full grid because learning rate and regularization matter on logarithmic scales and many interactions exist. Optuna, Ray Tune, or Weights & Biases Sweeps are suitable tools.

11.3 Shared search space

Parameter	Recommended search
Input window	28, 56, 84, 112
Batch size	64, 128, 256, 512
Learning rate	Log-uniform from `1e-5` to `3e-3`
Weight decay	Log-uniform from `1e-7` to `1e-2`
Dropout	0.0 to 0.4
Gradient clipping	0.5, 1.0, 2.0, 5.0
Scheduler	Plateau, cosine, cosine with warm-up
Loss	MAE, Huber, scaled/horizon-weighted variants
Lags	`[1,7,14,28]`, `[7,14,28,56]`, annual variants
Rolling windows	`[7,14,28]`, `[7,28,56]`

11.4 Transformer search space

Parameter	Candidate values
Embedding dimension	32, 64, 128, 256
Attention heads	2, 4, 8
Layers	1 to 4
Feed-forward multiplier	2x, 4x, 8x embedding size
Dropout	0.0 to 0.3
Historical attention	Causal versus unmasked
Decoder	Direct versus autoregressive

Embedding dimension must be divisible by the number of heads.

11.5 DeepAR search space

Parameter	Candidate values
Hidden size	32, 64, 128, 256
LSTM layers	1 to 3
Dropout	0.0 to 0.3
Likelihood	Gaussian, Poisson, negative binomial
Teacher-forcing ratio	0.5 to 1.0
Scaling	None, mean scale, robust scale, `log1p`

11.6 TFT search space

Parameter	Candidate values
Hidden dimension	32, 64, 128, 256
Attention heads	1, 2, 4, 8
LSTM layers	1 or 2
Feed-forward size	1x, 2x, or 4x hidden size
Dropout	0.0 to 0.3
Objective	MAE, Huber, quantile loss

11.7 Trial pruning and final confirmation

1. Give every trial a small minimum epoch budget.

2. Prune trials that are clearly worse than the median.
3. Continue promising trials to convergence.
4. Retrain the best configuration with three to five seeds.
5. Compare paired predictions across the same origins.
6. Report mean, standard deviation, confidence intervals, latency, and parameter count.

11.8 Objective selection

For competition alignment, use a fully correct hierarchical WRMSSE implementation. For a business deployment, model selection may combine forecast error with latency and asymmetric inventory costs. The statistically most accurate model is not automatically the best operational model.

12 Complete decision register

12.1 Data decisions

- Forecast item-store, item, category, department, or aggregate demand?
- Treat unavailable prices as missing, not sold, or zero?
- Encode event name and type or use a binary event indicator?
- Add item, store, category, department, state, and region embeddings?
- Include yearly lags and longer rolling windows?
- Scale each series, use global scaling, or apply `log1p`?
- Remove inactive history or preserve zeros?
- Treat future calendar and planned-price covariates as known inputs?

12.2 Validation decisions

- Use one chronological split or multiple rolling origins?
- Evaluate one final 28-day block or all overlapping windows?
- Select with MAE, RMSSE, WRMSSE, or inventory cost?
- Measure bottom-level accuracy only or the full hierarchy?
- How many random seeds are sufficient?
- What improvement is operationally meaningful?

12.3 Modeling decisions

- Direct or autoregressive forecasting?
- Deterministic or probabilistic output?
- Gaussian, Poisson, negative-binomial, or zero-inflated likelihood?
- One global model or category-specific models?
- Compact TFT or full TFT?
- Causal or unmasked historical attention?
- Enforce nonnegative forecasts with softplus, clipping, or count likelihood?
- Round outputs for reporting or retain continuous expected demand?

12.4 Operational decisions

- Maximum acceptable training and inference time?
- Retraining schedule?
- Forecast refresh schedule?
- Drift and degradation thresholds?
- Cold-start strategy?
- Promotion and stockout adjustment?
- Checkpoint/model registry and rollback?

- Experiment tracking and reproducibility standard?

13 Known limitations and next improvements

1. No reproduced data, logs, checkpoints, or result table are committed.
2. Hyperparameter tuning is manual rather than automated.
3. Early stopping is not implemented.
4. Configured warm-up is unused.
5. The optimizer setting is ignored; Adam is always used.
6. Feature-related YAML fields do not control preprocessing.
7. Full-period price normalization may leak future information.
8. Features and targets are not systematically scaled.
9. Known future covariates are not given to forecast decoders.
10. Item and store identity features are absent.
11. Point forecasts are not constrained to be nonnegative.
12. Gaussian DeepAR is imperfect for sparse count demand.
13. Probabilistic calibration is not evaluated.
14. Official hierarchical WRMSSE is not implemented.
15. The evaluation CLI currently reports only MAE and RMSE.
16. Overlapping test windows differ from the official single-origin M5 task.
17. Seasonal-naive and boosted-tree baselines are absent.
18. The default experiment covers one store, so conclusions do not automatically generalize across the M5 hierarchy.
19. Histories are persisted in HTML, but there is no database-backed experiment registry or machine-readable trial ledger.
20. Dataset indices are stored as Python tuples; scaling back to all stores would again consume substantial memory.

A sensible implementation roadmap is:

1. Make feature configuration operational and remove normalization leakage.
2. Add deterministic-algorithm controls and repeat runs across multiple seeds.
3. Add seasonal-naive and LightGBM baselines.
4. Implement rolling-origin validation and official WRMSSE.
5. Add feature/target scaling and identity embeddings.
6. Add future known covariates.
7. Implement automated tuning and true early stopping.
8. Add negative-binomial or quantile probabilistic forecasts.
9. Add a structured experiment ledger alongside the portable HTML report.
10. Compare accuracy against latency and business cost, then validate on additional stores.

14 Presentation structure

A recommended presentation flow is:

1. Business problem and inventory consequences.
2. Dataset, granularity, and 28-day horizon.
3. Feature-engineering rationale.
4. Chronological splitting and leakage prevention.
5. Baseline-first model-selection philosophy.
6. Architecture comparison.
7. Training and checkpoint selection.
8. Evaluation metrics and why MAPE is avoided.
9. Reproduced results when available.
10. Complexity-versus-accuracy tradeoff.
11. Limitations and next experiments.

12. Production plan and monitoring.

The safest current conclusion is:

> The project provides a tested, modular framework for comparing direct, autoregressive, probabilistic, and attention-based global forecasters on a consistent M5 pipeline. The architecture is ready for controlled experiments, but model-ranking claims should be reproduced with recorded multi-seed, time-series-aware validation and official hierarchical metrics before a final winner is declared.

15 Cross-question defense guide

15.1 Why use deep learning if a linear model is available?

The purpose is to measure whether nonlinear temporal interactions and long-range dependencies provide material improvement. The linear model remains the decision baseline. A complex model should be selected only if its improvement is consistent and economically meaningful.

15.2 Why not use ARIMA?

ARIMA can be strong for individual stationary series, but fitting tens of thousands of independent models is costly and prevents cross-series learning. Global models share statistical strength. A seasonal statistical baseline should nevertheless be included.

15.3 Why exclude XGBoost and LightGBM?

The implemented scope focuses on neural sequence architectures. This is not evidence that boosted trees are inferior. LightGBM is a strong candidate for a production benchmark on lag-engineered retail data.

15.4 Why use only 28 days of history?

It captures four weekly cycles and keeps memory and complexity controlled. It is an initial hypothesis. Proper tuning should compare 28, 56, 84, and 112 days.

15.5 Why choose 7-, 14-, and 28-day lags?

They represent one, two, and four-week seasonal references and correspond to recurring weekday shopping patterns.

15.6 Why not include a 365-day lag?

Annual seasonality may matter, but it requires sufficient history and may become stale when prices or assortment change. It is a valid experimental extension.

15.7 Why MAE instead of MSE?

MAE is directly interpretable in unit sales and less dominated by large spikes. RMSE is retained to expose severe misses.

15.8 Why not use MAPE?

Retail demand contains many zeros. Percentage error is undefined or unstable near zero, so scaled measures such as RMSSE are more reliable.

15.9 Why use Adam?

Adam is a robust adaptive optimizer for recurrent and attention networks and generally requires less manual scheduling than plain SGD.

15.10 Why might the encoder-decoder model win?

It combines cross-attention to historical context with explicit dependencies among future steps. It should only be called the winner after controlled reproduced comparisons.

15.11 What is teacher forcing?

During training, the decoder receives the true previous future value. During inference, it must receive its own previous prediction. This speeds training but creates exposure bias.

15.12 How is target leakage prevented?

Input windows occur strictly before their targets and all partitions are chronological. Validation and test receive only prior context. Full-period price normalization remains a concern that should be corrected or explicitly justified.

15.13 Why use a causal encoder mask?

It enforces forward-only historical representations. Since the full input history is known at the forecast origin, unmasked historical attention is also valid and should be tested.

15.14 How does the project represent uncertainty?

DeepAR predicts Gaussian mean and scale. The current evaluation uses only the mean, so prediction-interval and calibration evaluation remain future work.

15.15 Is Gaussian demand appropriate?

It is convenient and stable but not ideal for nonnegative, sparse, overdispersed counts. Negative-binomial or zero-inflated likelihoods are stronger candidates.

15.16 Is the TFT interpretable?

Its variable-selection and attention mechanisms can support interpretation, but the current implementation does not expose those weights. Interpretability is therefore potential rather than a completed output.

15.17 Does the model use known future holidays or prices?

No. It consumes covariates only inside the historical window. Adding known future calendar, SNAP, and planned-price inputs is a high-value improvement.

15.18 How should tuning avoid test leakage?

Use rolling validation folds entirely before the test period, choose the configuration on average validation performance, freeze all decisions, and evaluate the test once.

15.19 How can we know whether an improvement is statistically real?

Train multiple seeds, evaluate paired forecasts on identical series and origins, calculate confidence intervals, and use paired bootstrap or comparable paired-error analysis.

15.20 Would the lowest-MAE model automatically be deployed?

No. Deployment also considers WRMSSE, high-value products, inference latency, memory, uncertainty calibration, failure modes, and asymmetric stockout/overstock cost.

15.21 How would new products be handled?

Use product metadata, category/store embeddings, analogous-product history, and a category-level seasonal fallback. The present pipeline has no explicit cold-start mechanism.

15.22 How would drift be monitored?

Track rolling error by store, category, demand volume, and forecast horizon; monitor feature distributions and probabilistic interval coverage; investigate or retrain when sustained changes exceed thresholds.

15.23 What is the single most important next experiment?

Build a leakage-safe rolling-origin benchmark containing seasonal-naive, linear, LightGBM, and all neural candidates; evaluate multiple seeds with official hierarchical WRMSSE; and preserve every configuration, checkpoint, and result artifact.

16 Repository verification notes

At the time this guide was prepared:

- The `main` branch included the scoped CA_1 pipeline, consolidated report generator, and one-click Windows launcher through commit beb496b.
- All 29 automated tests passed; five non-failing PyTorch performance warnings were emitted for pre-normalized Transformer encoders.
- Default data scope was CA_1, latest 365 days, with a 28-day input and 28-day horizon.
- The repository itself contained no M5 CSVs or completed training artifacts because data and model outputs are Git-ignored.
- Approximate parameter counts with 19 input variables were:

Model	Parameters
Linear	14,924
Encoder-only Transformer	698,012
Encoder-decoder Transformer	1,392,001
DeepAR	407,810
Compact TFT	519,064

These parameter counts quantify an important decision tradeoff: the encoder-decoder Transformer has roughly twice the parameters of the encoder-only model and far more than the linear baseline. Any accuracy improvement should be evaluated against that added cost.